Nama : Rofiq Aristiyawan
NIM : 254107020060
Kelas/Absen : TI-1G/27

Percobaan 5.2.1

1. Pada bagian if menjadi base case, yaitu kondisi berhenti rekursi, pada code kondisi nya n == 1, lalu method akan mengembalikkan nilai 1. Lalu pada bagian else adalah proses rekursi, menghitung n * faktorialDC(n-1) sampai mencapai base case.
    - If = penghenti proses.
    - Else = proses penghitungan berulang secara rekursi.
2. Bisa, karena faktorialBF() adalah mengalikan angka 1 sampai n, maka selain for bisa menggunakan while atau do while. Pembuktian saya menggunakan while;

```
int faktorialBF(int n) {
  int fakto = 1;

  //for(int i=1; i<=n; i++) {
    //fakto = fakto * i;

  int i = 1;
  while(i <= n){
    fakto = fakto * i;
    i++;
  }
  return fakto;
}
```

3. Perbedaannya:

    fakto *= i:
    - Bentuk singkat dari fakto = fakto * I;
    - Dipakai pada iterasi/Brute Force.
    - Nilai fakto diperbarui terus setiap looping.
    - Proses bertahap 1,2,3,4,…,n.

    fakto = n * faktorialDC(n-1); :
    - Dipakai pada rekursi/Divide coquer.
    - Nilai fakto dihitung dari hasil pemanggilan method yang lebih kecil.
    - Tidak memakai loop, tetapi memanggil fungsi berulang.
4. Kesimpulannya:
    - faktorialBF() menghitung faktorial dengan perulangan langsung dari 1 sampai n.
    - faktorialDC() menghitung faktorial dengan rekursi, yaitu memecah masalah menjadi n * (n-1)!.
    - Brute Force lebih mudah dipahami karena alurnya lurus.
    - Divide and Conquer lebih elegan secara konsep, tapi butuh pemahaman rekursi dan base case.
    - Untuk kasus faktorial, hasil akhirnya sama, hanya mekanisme prosesnya yang beda.

Percobaan 5.3.1

1. Perbedaannya :

   pangkatBF() :
   - Menggunakan iterasi.
   - Mengalikan angka dasar sebanyak jumlah pangkat.
   - Misalnya 2^5 dihitung dengan 2 * 2 * 2 * 2 * 2.

   pangkatDC() :
   - Menggunakan rekursi divide and conquer.
   - Pangkat dibagi menjadi submasalah lebih kecil.
   - Kalau pangkat genap: hasil = pangkatDC(a, n/2) * pangkatDC(a, n/2).
   - Kalau pangkat ganjil: hasil = pangkatDC(a, n/2) * pangkatDC(a, n/2) * a.
2. Iya, sudah termasuk.

   Pada divide and conquer, ada 3 tahap, yaitu:
   - divide: membagi masalah
   - conquer: menyelesaikan submasalah
   - combine: menggabungkan hasil submasalah

   Di method pangkatDC(), bagian combine terlihat saat hasil dua rekursi dikalikan:

```
return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
```

   untuk yang ganjil :

```
  return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);
```

3. Tetap relevan, tapi tidak wajib. Karena method dengan parameter lebih fleksibel. Satu objek bisa dipakai menghitung nilai lain tanpa harus mengubah atribut objek. bagus kalau method ingin dibuat lebih umum. Bisa tanpa parameter. Kalau memanfaatkan atribut class langsung, methodnya menjadi :

```
int pangkatBF(int a, int n) {
    // int hasil = 1;
    // for (int i = 0; i < n; i++) {
    // hasil = hasil * a;
    // }
    // return hasil;

    int hasil = 1;
    for (int i = 0; i < pangkat; i++) {
      hasil = hasil * nilai;
    }
    return hasil;
  }
```

4. Kesimpulannya:
   - pangkatBF() bekerja dengan perkalian berulang secara langsung.
   - pangkatDC() bekerja dengan membagi persoalan pangkat menjadi submasalah lebih kecil.
   - Brute Force lebih sederhana dan mudah ditelusuri.
   - Divide and Conquer lebih cocok untuk menunjukkan konsep rekursi, walau pada kode masih melakukan pemanggilan rekursi ganda pada bagian yang sama.

Percobaan 5.4.1

1. Variabel mid dibutuhkan untuk membagi array menjadi dua bagian, yaitu kiri dan kanan. Ini inti dari divide and conquer.
   mid menentukan titik tengah array agar array bisa diproses menjadi:
   - kiri: dari l sampai mid
   - kanan: dari mid+1 sampai r

   Tanpa mid, array tidak bisa dibelah, jadi konsep divide and conquer-nya bubar.
2. Statement itu dipakai untuk:
   - menghitung jumlah elemen bagian kiri array (lsum)
   - menghitung jumlah elemen bagian kanan array (rsum)

   Ini adalah tahap conquer, yaitu menyelesaikan dua submasalah hasil pembagian sebelumnya. Keduanya lalu digabungkan pada tahap berikutnya.

3. Karena total seluruh array adalah gabungan dari:
   - jumlah bagian kiri
   - ditambah jumlah bagian kanan

   Maka perlu:

```
return lsum + rsum;
```

   Itu adalah tahap combine, yaitu menyatukan hasil dari dua subarray menjadi satu total akhir. Kalau tidak dijumlahkan, hasilnya cuma setengah.
4. Base case dari totalDC() adalah saat:

```
if (l == r) {
        return arr[l];
    }
```

   Artinya, ketika indeks kiri sama dengan indeks kanan, berarti array tinggal punya satu elemen. Karena hanya satu elemen, langsung dikembalikan tanpa dibagi lagi. Inilah titik berhenti rekursi.
5. Kesimpulannya:
   - totalDC() menjumlahkan elemen array dengan cara membagi array menjadi dua bagian terus-menerus.
   - Rekursi berhenti saat bagian array tinggal satu elemen.
   - Hasil sisi kiri dan kanan lalu dijumlahkan kembali.
   - Alur kerjanya mengikuti pola divide, conquer, combine.
   - Dibanding totalBF() yang iteratif, totalDC() lebih menunjukkan konsep rekursif dalam pengolahan array.

Latihan Pratikum

1. Kode program

```
package Josheet5.BruteForceDivideCoquer.Minggu5;

public class Mahasiswa {
    String nama;
    String nim;
    int tahunMasuk;
```

```java
    int nilaiUTS;
    int nilaiUAS;

    public Mahasiswa(String nama, String nim, int tahunMasuk, int nilaiUTS, int
nilaiUAS) {
        this.nama = nama;
        this.nim = nim;
        this.tahunMasuk = tahunMasuk;
        this.nilaiUTS = nilaiUTS;
        this.nilaiUAS = nilaiUAS;
    }
}
```

```java
package Josheet5.BruteForceDivideCoquer.Minggu5;

public class NilaiMahasiswa {
    Mahasiswa[] dataMahasiswa;

    public NilaiMahasiswa(Mahasiswa[] dataMahasiswa) {
        this.dataMahasiswa = dataMahasiswa;
    }

    public int maxUTS(int left, int right) {
        if (left == right) {
            return dataMahasiswa[left].nilaiUTS;
        }

        int mid = (left + right) / 2;
        int maxKiri = maxUTS(left, mid);
        int maxKanan = maxUTS(mid + 1, right);

        return Math.max(maxKiri, maxKanan);
    }

    public int minUTS(int left, int right) {
        if (left == right) {
            return dataMahasiswa[left].nilaiUTS;
        }

        int mid = (left + right) / 2;
        int minKiri = minUTS(left, mid);
        int minKanan = minUTS(mid + 1, right);

        return Math.min(minKiri, minKanan);
    }
```

```java
    public double rataRataUAS() {
        int total = 0;
        for (int i = 0; i < dataMahasiswa.length; i++) {
            total += dataMahasiswa[i].nilaiUAS;
        }
        return (double) total / dataMahasiswa.length;
    }

    public String mahasiswaNilaiTertinggi(int nilaiTertinggi) {
        for (int i = 0; i < dataMahasiswa.length; i++) {
            if (dataMahasiswa[i].nilaiUTS == nilaiTertinggi) {
                return dataMahasiswa[i].nama;
            }
        }
        return "-";
    }


    public String mahasiswaNilaiTerendah(int nilaiTerendah) {
        for (int i = 0; i < dataMahasiswa.length; i++) {
            if (dataMahasiswa[i].nilaiUTS == nilaiTerendah) {
                return dataMahasiswa[i].nama;
            }
        }
        return "-";
    }
}
```

```java
package Josheet5.BruteForceDivideCoquer.Minggu5;

public class MainNilaiMahasiswa {
    public static void main(String[] args) {
        Mahasiswa[] mahasiswa = {
            new Mahasiswa("Ahmad", "220101001", 2022, 78, 82),
            new Mahasiswa("Budi", "220101002", 2022, 85, 88),
            new Mahasiswa("Cindy", "220101003", 2021, 90, 87),
            new Mahasiswa("Dian", "220101004", 2021, 76, 79),
            new Mahasiswa("Eko", "220101005", 2023, 92, 95),
            new Mahasiswa("Fajar", "220101006", 2020, 88, 85),
            new Mahasiswa("Gina", "220101007", 2023, 80, 83),
            new Mahasiswa("Hadi", "220101008", 2020, 82, 84)
        };

        NilaiMahasiswa nm = new NilaiMahasiswa(mahasiswa);
```

```java
        int utsTertinggi = nm.maxUTS(0, mahasiswa.length - 1);
        int utsTerendah = nm.minUTS(0, mahasiswa.length - 1);
        double rataUAS = nm.rataRataUAS();

        System.out.println("Nilai UTS Tertinggi (Divide and Conquer): " +
utsTertinggi);
        System.out.println("Mahasiswa dengan nilai UTS tertinggi: " +
nm.mahasiswaNilaiTertinggi(utsTertinggi));

        System.out.println("Nilai UTS Terendah (Divide and Conquer): " +
utsTerendah);
        System.out.println("Mahasiswa dengan nilai UTS terendah: " +
nm.mahasiswaNilaiTerendah(utsTerendah));

        System.out.println("Rata-rata nilai UAS (Brute Force): " + rataUAS);
    }
}
```

Hasil :

```
PS C:\Users\Qifor\Documents\Polinema\Semester 2\PratikumAlgoritmaStrukturDa
ta\PratikumAlgoritmaStrukturData>  & 'C:\Program Files\Java\jdk-25.0.2\bin\
java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp
' 'C:\Users\Qifor\AppData\Roaming\Code\User\workspaceStorage\99c46d2dd1f3d6
e763c1fb6295ab4410\redhat.java\jdt_ws\PratikumAlgoritmaStrukturData_eada4ea
6\bin' 'Josheet5.BruteForceDivideCoquer.Minggu5.MainNilaiMahasiswa'
Nilai UTS Tertinggi (Divide and Conquer): 92
Mahasiswa dengan nilai UTS tertinggi: Eko
Nilai UTS Terendah (Divide and Conquer): 76
Mahasiswa dengan nilai UTS terendah: Dian
Rata-rata nilai UAS (Brute Force): 85.375
PS C:\Users\Qifor\Documents\Polinema\Semester 2\PratikumAlgoritmaStrukturDa
ta\PratikumAlgoritmaStrukturData>
```

Run: MainP...
Run: MainS...
Run: MainN...